

INFORME TÉCNICO DE VALIDACIÓN Y ASEGURAMIENTO DE LA CALIDAD (QA)

Evaluación Integral del Sistema de Descuentos Automatizado mediante Pruebas Dinámicas y Unitarias

Módulo Evaluado:	Calculadora Interactiva de Descuentos (Proyecto Integrador)	Fecha de Reporte:	20 de mayo de 2026
Institución:	Universidad Politécnica Estatal del Carchi (UPEC)	Entorno Operativo:	PyCharm IDE / PowerShell Core
Equipo de Desarrollo:	Jhon Mavisoy, Kevin Anchundia, Milena Narvaez, Alex Moreno		

1. Arquitectura de Validación del Software

El proceso de aseguramiento de calidad se estructuró en dos fases complementarias para cubrir tanto el comportamiento funcional del software en tiempo de ejecución (Caja Negra) como la integridad estructural de sus subrutinas y manejo de excepciones (Caja Blanca). El componente central evaluado corresponde al módulo de cálculo matemático implementado en `calculadora.py`.

Para proveer un análisis riguroso, se diseñó un flujo que abarca desde la interfaz interactiva de consola expuesta al usuario final, hasta suites automatizadas bajo la librería estándar `unittest` de Python y la biblioteca complementaria de aserciones extendidas `testtools`. La generación de entradas fue robustecida mediante la simulación de datos pseudoaleatorios controlados provistos por `Faker`.

2. Fase I: Análisis de Caja Negra (Consola Interactiva)

Se realizó una auditoría de comportamiento directamente sobre el archivo de ejecución en caliente del sistema. El objetivo de esta fase fue comprobar la interactividad de la interfaz, el formateo de datos flotantes a cobrar y el disparo correcto de alertas de entrada cuando los valores violan las restricciones comerciales básicas del modelo de negocio (según los registros obtenidos en la **Evidencia 01**).

Registro de Interacción y Captura de Consola (Evidencia 01):

```
=====
Calculadora de Descuentos Interactiva
=====
Escribe 'salir' en cualquier momento para terminar.

Ingresa el precio original: 300
Ingresa el porcentaje de descuento: 20
>>> Precio final a cobrar: 240.0

Ingresa el precio original: 300
Ingresa el porcentaje de descuento: -5
>>> [ERROR]: El descuento debe estar entre 0 y 100

Ingresa el precio original: 430
Ingresa el porcentaje de descuento: 31
>>> Precio final a cobrar: 296.7
```

Análisis de Casos en Caliente:

Como se constata en el registro de flujos interactivos, el software responde con precisión ante tres escenarios críticos del negocio:

Escenario de Entrada	Valores Inyectados	Comportamiento Observado	Dictamen Técnico
Caso de Éxito Estándar	Precio: 300 Desc: 20	Retorna un flotante puro de 240.0	CORRECTO Deducción exacta del 20%.
Inyección Extralímite Negativa	Precio: 300 Desc: -5	Captura el flujo e imprime un string de [ERROR]	CORRECTO Control preventivo antes del cálculo.
Cálculo con Decimales Flotantes	Precio: 430 Desc: 31	Retorna un redondeo preciso a 296.7	CORRECTO Manejo exacto de precisión matemática.

3. Fase II: Automatización de Pruebas Unitarias Retrocompatibles

Para garantizar la portabilidad y la integración continua del código, se implementó una suite automatizada estructurada mediante clases que heredan de `testtools.TestCase` y se procesan nativamente sobre la arquitectura estándar de Python.

3.1. Ejecución Base de Pruebas Unitarias (Evidencia 02):

La primera corrida del motor se ejecutó a nivel de comando directo sobre el archivo de pruebas base para asegurar la compatibilidad sin dependencias externas complejas. El reporte devolvió estabilidad absoluta (4 pruebas ejecutadas con éxito en 0.135 segundos):

```
PS C:[Ruta del Proyecto]> python -m unittest test_calculadora.py -v
test_descuento_estandar_exito
(test_calculadora.TestCalculadoraDescuentos.test_descuento_estandar_exito) ... ok
test_error_descuento_fuera_de_rango
(test_calculadora.TestCalculadoraDescuentos.test_error_descuento_fuera_de_rango) ... ok
test_error_precio_no_positivo
(test_calculadora.TestCalculadoraDescuentos.test_error_precio_no_positivo) ... ok
test_super_descuento_con_bono
(test_calculadora.TestCalculadoraDescuentos.test_super_descuento_con_bono) ... ok

-----
Ran 4 tests in 0.135s

OK
```

3.2. Ejecución Profunda con Aislamiento de Casos y Faker (Evidencia 03):

Posteriormente, se expandió la suite hacia un entorno de auditoría profunda agregando pruebas iterativas de estrés alimentadas por la librería Faker con una semilla de control (Seed: 42), además de estructurar un bloque modular con soporte para SubTests. La traza detallada arrojó un desglose de 5 pruebas completadas de forma satisfactoria:

```
=====
AUDITORÍA DE QA: DESGLOSE DETALLADO DE PRUEBAS UNITARIAS
=====
test_analisis_valores_frontera_subtests
(__main__.TestCalculadoraDescuentosProfundo.test_analisis_valores_frontera_subtests)
__main__.TestCalculadoraDescuentosProfundo.test_analisis_valores_frontera_subtests ... ok
test_descuento_estandar_exito_masivo
(__main__.TestCalculadoraDescuentosProfundo.test_descuento_estandar_exito_masivo)
__main__.TestCalculadoraDescuentosProfundo.test_descuento_estandar_exito_masivo ... ok
test_error_descuento_fuera_de_rango
(__main__.TestCalculadoraDescuentosProfundo.test_error_descuento_fuera_de_rango)
__main__.TestCalculadoraDescuentosProfundo.test_error_descuento_fuera_de_rango ... ok
test_error_precio_no_positivo
(__main__.TestCalculadoraDescuentosProfundo.test_error_precio_no_positivo)
__main__.TestCalculadoraDescuentosProfundo.test_error_precio_no_positivo ... ok
test_super_descuento_con_bono
(__main__.TestCalculadoraDescuentosProfundo.test_super_descuento_con_bono)
__main__.TestCalculadoraDescuentosProfundo.test_super_descuento_con_bono ... ok

-----
Ran 5 tests in 0.148s

OK
```

4. Mapeo Analítico y Desglose de Responsabilidades de QA

Para un correcto entendimiento de las métricas anteriores, se detalla el rol de cada biblioteca en la suite profunda:

- **Motor Principal (Unittest):** Responsable del orden secuencial, el conteo total de casos de prueba (Ran 5 tests) y de emitir la bandera final de estado OK. Coordina la ejecución de los bucles de aislamiento mediante el uso nativo de `self.subTest()`.
- **Estrategia de Robustez (TestTools):** Intercepta de forma segura el ciclo de vida de los casos críticos de error (`test_error_descuento_fuera_de_rango` y `test_error_precio_no_positivo`). Provee las aserciones avanzadas que validan que el software dispare la excepción exacta `ValueError` sin colapsar el hilo de ejecución principal.
- **Materia Prima Sintética (Faker):** Alimenta la función `test_descuento_estandar_exito_masivo` generando matrices dinámicas de números flotantes y enteros para simular cargas transaccionales repetitivas en fracciones de segundo (procesando la suite completa en apenas 0.148s).

Garantía de Código de Salida:

Ambas corridas automatizadas finalizaron con el estado nativo OK y un código de salida limpio (Exit Code 0). Esto certifica que el módulo evaluado carece de fugas lógicas o desbordamientos matemáticos bajo los conjuntos de datos simulados.

5. Conclusiones y Recomendaciones

1. **Sólido Control de Entrada:** El software demuestra consistencia absoluta tanto en su capa visual (interfaz de comandos de la calculadora de la Fase I) como en su núcleo lógico analizado en las unidades de testing de la Fase II, rechazando proactivamente valores aberrantes o negativos.
2. **Estabilidad para Despliegue:** La suite demuestra niveles óptimos de eficiencia en tiempo de procesamiento (promedio de 0.14s para ejecuciones masivas). La arquitectura híbrida elegida es ideal para ser acoplada en un pipeline de integración continua (CI) para el resguardo de futuros módulos del sistema.

Fin del Reporte Integral de Auditoría de Código — Módulo Calculadora interactiva